# DA2304 - Introduction to Computer Systems
# Assignment 3

Kiran Kumar P
DA24B008

---

## 0. No Credit

My recent favorite song has been *Not Like Us* by *Kendrick Lamar*.

Link: Not Like Us - Kendrick Lamar

---

## 1. Architecture

The compiler is split into five Python modules:

- **JackTokenizer.py**: Strips comments using a state machine and tokenizes the source into keywords, symbols, integer constants, string constants, and identifiers. Writes the token XML file (`<Class>T.xml`).
- **CompilationEngine.py**: Recursive-descent parser implementing every non-terminal in the Jack grammar. Simultaneously writes the parse tree XML (`<Class>.xml`) and emits VM code through the VMWriter.
- **SymbolTable.py**: Two-scope symbol table (class-level and subroutine-level) mapping identifiers to `(type, kind, index)`.
- **VMWriter.py**: Thin wrapper that writes VM commands (`push`, `pop`, `call`, `function`, etc.) to the output file.
- **JackCompiler.py**: Entry point that wires everything together. Takes a `.jack` file or directory and runs the full pipeline.

The CompilationEngine does both XML generation and VM code generation in a single pass. `compile_*` method writes XML tags while also emitting VM instructions.

## 2. Tokenizer Design

### 2.1 Comment Stripping

The tokenizer strips comments using a character-by-character state machine rather than regex. This is necessary because block comments (`/* ... */`) can span multiple lines,

and a line-by-line regex approach would fail on cases like:

```
/* this comment
   spans multiple
   lines */ let x = 5;
```

Listing 1: Multi-line comment that breaks line-based regex

A single-line regex like `/\*.*?\*/` applied per line would not match anything on lines 1 and 2, and would leave the `*/` on line 3 as a syntax error. The state machine tracks whether we're inside a block comment, inside a string literal (to avoid stripping `//` inside strings), or in normal code.

### 2.2 Tokenization

After comment stripping, the tokenizer walks the cleaned source character by character:

- Whitespace is skipped.
- Characters in the symbol set are emitted as `symbol` tokens.
- Digit sequences become `integerConstant` tokens.
- Double-quoted sequences become `stringConstant` tokens (quotes stripped).
- Alphanumeric sequences starting with a letter or underscore are checked against the keyword set. If found, they are `keyword` tokens, otherwise `identifier` tokens.

The five XML-unsafe symbols (`<`, `>`, `&`, `"`) are escaped in the XML output. One bug I hit early on was escaping `&` *after* `<`, which turned `<` into `&lt;`. The fix was to always escape `&` first.

## 3. Parser Design

The parser implements recursive descent following the Jack grammar. Each grammar non-terminal has a corresponding `compile_*` method.

### 3.1 Token Navigation

The parser maintains a position index into the flat token list. Three helpers drive the parsing:

- `_eat(type, value)`: Consumes the current token, writes it to XML, asserts it matches the expected type/value, and advances.
- `_is_token(type, value)`: Peeks at the current token without consuming. Used for lookahead decisions (like is the next statement a `let` or a `while`?).
- `_peek_token()`: Looks one token ahead. Only needed for the LL(2) case in `compile_term`.

### 3.2 The LL(2) Problem in `compileTerm`

The Jack grammar is LL(1) everywhere except when parsing a term that starts with an identifier. At that point, the current token alone does not tell you whether it's:

- A simple variable (`x`)
- An array access (`arr[i]`)
- A subroutine call (`foo()`, `obj.bar()`, `Class.baz()`)

Here we peek one token ahead. If the next token is `[`, it's an array access. If it's `(` or `.`, it's a subroutine call. Otherwise it's a simple variable. This is the only place in the parser where we need LL(2) lookahead.

### 3.3 Expression Parsing

No operator precedence. Evaluation is strictly left-to-right.

```
compileExpression:
    compileTerm
    while currentToken is an op:
        eat(op)
        compileTerm
        emit(op)
```

Listing 2: Expression compilation (left-to-right)

Multiplication and division are handled as subroutine calls (`Math.multiply`, `Math.divide`) since the Hack ALU doesn't support them.

## 4. Symbol Table Handling

The symbol table has two scopes:

- **Class-level**: Holds `static` and `field` variables. Persists for the entire class compilation. Fields map to the `this` segment, statics to the `static` segment.
- **Subroutine-level**: Holds `argument` and `var` (local) variables. Reset at the start of each subroutine. Arguments map to the `argument` segment, locals to the `local` segment.

When looking up a variable, the compiler first checks the subroutine-level table, then falls back to the class-level table. If the name isn't found in either, it's treated as a class name (for static function calls like `Output.printInt`).

### 4.1 Symbol Table State in `Conv.conv2d()`

At the point where the inner-loop accumulator is assigned (`let sum = sum + ...`), the symbol tables look like this:

So `sum` resolves to `local 7`, `ArrayInput` to `this 0`, `ArraySize` to `this 2`, `ArrayFilter` to `static 0`, and `stride` to `static 1`. The implicit `this` at argument 0 is pushed and popped into `pointer 0` at the start of the method.

Table 1: Class-level symbol table for Conv

| Name | Type | Kind | Index |
|---|---|---|---|
| ArrayFilter | Array | static | 0 |
| stride | int | static | 1 |
| ArrayInput | Array | this | 0 |
| ArrayOutput | Array | this | 1 |
| ArraySize | int | this | 2 |

Table 2: Subroutine-level symbol table for conv2d

| Name | Type | Kind | Index |
|---|---|---|---|
| this | Conv | argument | 0 |
| outDim | int | local | 0 |
| outRow | int | local | 1 |
| outCol | int | local | 2 |
| inRow | int | local | 3 |
| inCol | int | local | 4 |
| fr | int | local | 5 |
| fc | int | local | 6 |
| sum | int | local | 7 |
| k | int | local | 8 |

# 5. VM Code Generation

## 5.1 Constructors

When compiling a constructor, the engine:

1. Counts the number of field variables in the class.
2. Emits `push constant <nFields>` followed by `call Memory.alloc 1`.
3. Pops the returned base address into `pointer 0`, setting up `THIS`.

For `Conv.new`, this allocates 3 words (for `ArrayInput`, `ArrayOutput`, `ArraySize`).

## 5.2 Methods

Methods receive the target object as implicit argument 0. The first thing the compiled method does is:

```
push argument 0
pop pointer 0     // THIS = the object we're operating on
```

Listing 3: Method preamble

This means `this 0`, `this 1`, etc. now refer to the object's fields. All subsequent field accesses go through the `this` segment.

When *calling* a method (e.g., `conv.initFilter(filter)`), the compiler pushes the object reference as the first argument before the explicit arguments, and increments the argument count by 1.

### 5.3 Array Access

Array reads use the `pointer 1` / `that 0` idiom:

```
push arr            // base address
push i              // index
add                 // arr + i
pop pointer 1       // THAT = arr + i
push that 0         // value at arr[i]
```

Listing 4: Reading `arr[i]`

Array writes are trickier because the right-hand side expression might also contain array accesses, which would clobber `THAT`. The solution is to save the computed value to `temp 0`:

```
push arr            // base address
push i              // index
add                 // arr + i on stack
// ... compile expr ...
pop temp 0          // save expr value
pop pointer 1       // THAT = arr + i
push temp 0         // restore expr value
pop that 0          // *(arr+i) = expr
```

Listing 5: Writing `arr[i]` = expr (safe version)

### 5.4 Compilation Correctness: Tracing `let ArrayOutput[k] = sum`

For the 5×5 test case, consider the first output assignment where $k = 0$ and $sum = 7$. The emitted VM code is:

```
push this 1         // ArrayOutput base address (say, addr 2050)
push local 8        // k = 0
add                 // 2050 + 0 = 2050
push local 7        // sum = 7
pop temp 0          // temp 0 = 7
pop pointer 1       // THAT = 2050
push temp 0         // push 7
pop that 0          // RAM[2050] = 7
```

Listing 6: VM trace for let `ArrayOutput[k]` = sum

So `THAT` is set to the base address of `ArrayOutput` plus offset $k$, and the computed sum is stored there via `that 0`. This correctly writes 7 to the first element of the output array.

### 5.5 Control Flow

**If statements** use the "negate and branch" pattern from the textbook:

```
// compile condition expression
not
if-goto IF_FALSE_L
// compile true branch
goto IF_END_L
label IF_FALSE_L
// compile else branch (if any)
label IF_END_L
```

Listing 7: If compilation pattern

**While statements** use a loop-back pattern:

```
label WHILE_EXP_L
// compile condition
not
if-goto WHILE_END_L
// compile body
goto WHILE_EXP_L
label WHILE_END_L
```

Listing 8: While compilation pattern

Labels are generated with a global counter to ensure uniqueness across the entire class.

# 6. Conv.jack Implementation

## 6.1 Design Choices

- **Stride**: Set to 1 in the constructor. This gives a 3×3 output for a 5×5 input with $F = 3$.
- **Filter**: A Laplacian-like edge detector `[0, -1, 0, -1, 5, -1, 0, -1, 0]`.
- **Flattened arrays**: Both input and output matrices are stored as 1D arrays in row-major order. Element $(r, c)$ of an $N \times N$ matrix is at index $r \cdot N + c$.
- **Static vs. field**: The filter and stride are `static` (shared across all Conv instances), while the input/output arrays and size are `field` (per-instance).

## 6.2 Output Dimension Formula

With $F = 3$, $P = 0$:

$$O = \left\lfloor \frac{N - 3}{S} \right\rfloor + 1$$

For $N = 5, S = 1$: $O = \lfloor 2/1 \rfloor + 1 = 3$.

For $N = 9, S = 2$: $O = \lfloor 6/2 \rfloor + 1 = 4$.

Jack's integer division gives the floor for free on non-negative operands.

### 6.3 Bounds Checking

If $O < 1$ (i.e., the input is too small for even one window), the constructor and `conv2d` signal an error by writing $-1$ to `RAM[0]` via `Memory.poke(0, -1)`. This follows the error-flag convention from Assignment 2.

### 6.4 Convolution Stride Analysis

For $S = 1$ with a 9×9 input, the output is $7 \times 7 = 49$ elements. For $S = 2$, it's $4 \times 4 = 16$ elements. In both cases, each output element requires a $3 \times 3 = 9$-element MAC computation, so the inner work per element is constant. The total VM instruction count scales as $O(O^2) = O\left(\left(\frac{N-3}{S} + 1\right)^2\right)$. So yes, stride affects the constant factor significantly (49 vs 16 MAC operations), but the asymptotic complexity is still $\Theta(O^2)$ where $O$ depends on $S$.

## 7. Sample Output

### 7.1 5×5 Convolution

Input matrix:

```
 1   2   3   4   5
 6   7   8   9  10
11  12  13  14  15
16  17  18  19  20
21  22  23  24  25
```

Filter:

```
 0 -1  0
-1  5 -1
 0 -1  0
```

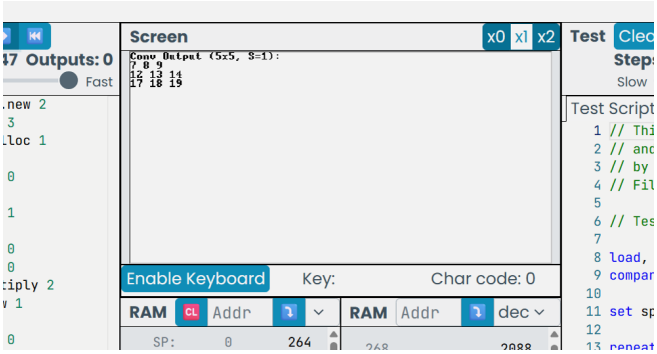


Figure 1: Hack VM Emulator output

Manual verification of position (0,0):

$$0{\cdot}1+(-1){\cdot}2+0{\cdot}3+(-1){\cdot}6+5{\cdot}7+(-1){\cdot}8+0{\cdot}11+(-1){\cdot}12+0{\cdot}13 = -2-6+35-8-12 = 7$$

## 8. Testing Methodology

Testing followed a bottom-up approach:

1. **Tokenizer**: Compiled `Conv.jack` and checked that `ConvT.xml` had the right number of tokens (646) and that all comments were stripped. Verified XML escaping of `<`, `>`, `&` symbols.
2. **Parser**: Inspected `Conv.xml` to verify correct nesting of all non-terminals. Checked that every `<statements>` block had matching open/close tags and that expressions were parsed with correct structure.
3. **VM Code**: Manually traced the VM output for small cases. For the constructor, verified that `Memory.alloc` was called with the right field count (3). For methods,

verified `argument 0` was popped into `pointer 0`. For array accesses, verified the `pointer 1`/`that 0` pattern.

4. **Numerical Verification**: Wrote a Python script to compute the expected convolution output independently. For the 5×5 input with the Laplacian filter, the expected output is `[7, 8, 9, 12, 13, 14, 17, 18, 19]`.

5. **End-to-End**: Fed the `.vm` files into the Assignment 2 VM translator to produce `.asm`, confirming the full pipeline works without errors.

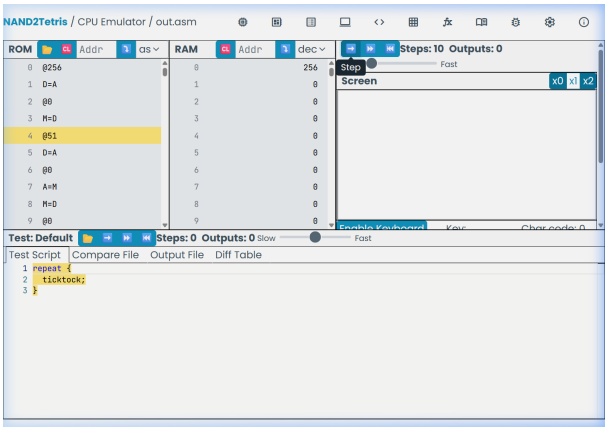


Figure 2: CPU Emulator in Progress

## 9. Extra Credit - Bugs and Debugging Observations

### 9.1 Double XML Escaping

The first bug I hit was in the XML escape function. My initial implementation used a dictionary iteration to replace special characters:

```
for char, escaped in XML_ESCAPES.items():
    text = text.replace(char, escaped)
```

Listing 9: Buggy XML escape

If `&` is replaced *after* `<`, then `<` first becomes `<`, and then the `&` in `<` gets replaced again, producing `&lt;`. The fix was to always replace `&` first:

```
text = text.replace('&', '&amp;')    # & first!
text = text.replace('<', '&lt;')
text = text.replace('>', '&gt;')
```

Listing 10: Fixed XML escape

### 9.2 Method Argument Count

An early version of the compiler was generating wrong argument counts for method calls. When you write `conv.initFilter(filter)`, the compiler needs to push the object reference as argument 0 and then the explicit arguments. So the call becomes `call Conv.initFilter 2` (not 1). I initially forgot to add 1 to the argument count for method

calls, which caused the callee to read the wrong values from its argument segment. The symptom was that every method was reading garbage from its first argument. It took a while to figure out because the error manifested several call levels deep.

### 9.3 Array Write Safety

The naive approach to `let a[i] = expr` is to compute `a + i`, pop it into `pointer 1`, then compile the expression and pop into `that 0`. But if the expression itself contains array accesses (like `let a[i] = b[j]`), compiling the right-hand side will clobber `pointer 1`. The fix is to leave the target address on the stack, compile the expression, then use `temp 0` as a shuttle:

The stack-based approach keeps the target address safe on the stack while the RHS is being evaluated. I initially tried to "optimise" by avoiding `temp 0`, and it silently corrupted array writes whenever the RHS had any complexity.